

Module 4 — Knowledge Representation

This is the knowledge-representation module of a knowledge base on production enterprise AI; it can be understood on its own.

Why this module exists: Vector RAG is great at "find me passages about X." It's weak at "how are these entities related?" and "answer a question that requires traversing connections." Structured knowledge — knowledge graphs, ontologies, semantic layers — encodes entities and their relationships explicitly, giving AI systems something precise to reason over. This module also carries the semantic-layer thread of the system-of-record theme: how messy enterprise schemas become something an agent can actually reason about.

This module covers seven topics. First, knowledge graphs — their nodes, edges, and triples. Second, ontologies versus taxonomies. Third, schema design. Fourth, entity resolution. Fifth, graph databases and query languages, such as Neo4j and Cypher. Sixth, the semantic layer over enterprise data, which is the third strand of the system-of-record theme. And seventh, how structured knowledge integrates with LLMs.

4.1 Knowledge graphs: nodes, edges, and triples

Why it matters. A knowledge graph represents information as entities, called nodes, connected by relationships, called edges, often with properties on both. It captures facts as connections. For instance, one relationship might say that ACME has an opportunity called Renewal-2026; another might say that Renewal-2026 includes the Firewall-Bundle. Where a document says things in prose and a table says things in rows, a graph says things as an explicit web of relationships you can traverse and query precisely. It's how you answer "connect-the-dots" questions and how you give an LLM a source of structured truth.

Here's an analogy. A vector store is like a pile of well-indexed articles. A knowledge graph is like the org chart plus the wiring diagram — it doesn't just have the facts, it has how they connect, so you can trace paths such as who reports to whom, or which product depends on which component.

How it works. - A node is an entity: a customer, product, contract, person, or incident. - An edge is a typed, directed relationship, such as OWNS, DEPENDS_ON, or RENEWS. - A triple is the atomic unit in the RDF tradition. It links a subject, a predicate, and an object — for example, ACME has the contract C-1023. A graph is a set of triples. - Properties are attributes on nodes and edges: a contract's value, or an edge's "since" date. The Labeled Property Graph model, used by Neo4j, is built around rich properties, whereas RDF is built around triples plus formal semantics. - Provenance means good knowledge graphs record where each fact came from, including source and timestamp. This is essential for trust and auditability.

The main options — two traditions. - The RDF and W3C stack uses triples, the SPARQL query language, and OWL and RDFS for formal ontologies. It's strong for standards-based, interoperable, reasoning-heavy domains such as life sciences and government. - The Labeled Property Graph model uses nodes and edges with properties, queried with Cypher or Gremlin in Neo4j and similar systems. It's more developer-friendly and dominant in enterprise application contexts.

When to use it (and when not to). Use a knowledge graph when relationships are first-class — think recommendations, dependencies, fraud rings, org and asset hierarchies, or supply chains. Use it when you need multi-hop reasoning, when you must integrate heterogeneous sources under one model, or when explainability matters, since a graph path is an auditable explanation. Avoid the overhead when your questions are "find similar text" — that's vector RAG.

Where it goes wrong. Common failures include building a graph nobody queries, which is modeling for its own sake; underestimating the construction and maintenance cost; producing noisy graphs from bad extraction or entity resolution; and treating a knowledge graph as a substitute for, rather than a complement to, vector search.

A concrete example. A knowledge graph links accounts to opportunities to quotes to products to components to known issues. The question "Which renewing accounts depend on a component with an open critical defect?" is one graph traversal — a question vector RAG can only approximate by luck.

4.2 Ontologies versus taxonomies

Why it matters. Both organize concepts, but at different power levels. A taxonomy is a hierarchy — a tree of categories such as Product, then Networking, then Firewall, then FW-9000. An ontology is a richer model of concepts and the relationships and rules among them. It might say that a firewall is a security product, has throughput, is compatible with certain licenses, and cannot be sold without a support plan. Getting this vocabulary right matters because the ontology is the shared meaning an AI agent reasons over — it's the difference between "categories" and "a model of the domain."

How it works. - A taxonomy is parent-and-child classification: "is-a" and "part-of" hierarchies. It's simple and powerful for navigation and faceting. - An ontology has classes; properties and relationships, with domains and ranges; constraints and axioms; and sometimes inference rules — for instance, if X is a Y and Y has property P, then X has P. It defines what can exist and how it can relate. It's expressed in OWL or RDFS in the RDF world, or as a schema or metamodel in the Labeled Property Graph world. - A controlled vocabulary or thesaurus sits between the two: agreed terms plus synonyms and relations, expressed in SKOS. - Schema versus instances: the ontology is the template — Customer, Order, "places" — while the graph holds the instances, such as ACME places Order-42.

The main options — tools. Protégé is used for OWL editing, and SKOS for vocabularies. In Labeled Property Graph land the ontology is often a documented schema plus constraints. There are also enterprise catalogs and data-modeling tools, and increasingly, LLMs assist in drafting ontologies from documents.

When to use it (and when not to). A taxonomy suffices for classification and navigation. Invest in an ontology when you need shared semantics across systems, machine reasoning or validation, or an agent that must understand rules — for example, that a quote can't include incompatible products. Start minimal: a lightweight ontology you maintain beats an elaborate one you don't.

Where it goes wrong. Watch for over-engineering, meaning academic ontologies that never ship; analysis paralysis; ontologies that drift from reality because no one owns them; and conflating a taxonomy with an ontology and then being surprised it can't express rules.

A concrete example. A configure-price-quote domain ontology defines Product, Bundle, Feature, Constraint, and PriceRule, along with relationships like REQUIRES, INCOMPATIBLE_WITH, and UPSELLS_TO. An agent uses it to reason that a proposed configuration violates a REQUIRES constraint before it ever generates a quote.

4.3 Schema design

Why it matters. Schema design is deciding what your nodes, edges, and properties are — the model of your domain. Good schema makes queries simple and answers correct; bad schema makes everything downstream painful. For AI, the schema is also what the agent "reads" to know how to query — so clarity is a functional requirement, not just hygiene.

How it works — mechanics and principles. - Model relationships as edges, not as foreign keys buried in properties. The point of a graph is traversable connections. - Use nouns as nodes and verbs as edges as a first cut. Promote a relationship to a node when it has its own attributes or relationships. A "Purchase" with a date, an amount, and links to Product and Rep is often better as a node — this is called reification. - Use consistent naming conventions with human-readable labels; name edges as verbs or relationships, such as WORKS_FOR rather than "rel1." LLMs generate better queries against clear names. - Get granularity right: too coarse loses queryability, too fine explodes complexity. - Handle provenance and temporality: timestamp facts, and model change over time when it matters, as with contracts, prices, and org changes. - Align to the source system of record. Where the graph mirrors CRM or ERP entities, mirror their identity and keys so you can join back.

When to use it (and when not to). Design for the questions you must answer — this is query-driven modeling — not for theoretical completeness. Iterate: start with the top ten questions, model just enough, and expand as needed.

Where it goes wrong. Pitfalls include modeling everything as properties, which kills traversal; inconsistent naming, which breaks both humans and LLM query-generation; having no identity strategy, which leads to duplicate entities; ignoring time, so you can't answer "what was true then?"; and a schema that no query ever uses.

A concrete example. A team initially stores a rep's ID as a property on Account. Later they need "all deals two reps collaborated on" — impossible cleanly. Consider a pattern where two reps each have a WORKS_ON relationship to the same Opportunity. Refactoring to that shared-opportunity pattern makes the traversal trivial.

4.4 Entity resolution

Why it matters. Entity resolution — also known as deduplication, record linkage, or identity resolution — is determining when different records refer to the same real-world entity. For instance, "ACME Corp," "Acme Corporation," and "ACME Inc." resolve to one company. It's the make-or-break step for any knowledge graph or unified data layer: without it, the graph is full of duplicate nodes and the "connections" are fragmented and wrong. It's also one of the hardest, messiest problems in enterprise data.

How it works. - Blocking cheaply groups plausibly-matching records so you avoid comparing everything to everything, since comparing every pair is infeasible at scale. - Matching compares candidate pairs. This can use deterministic rules, such as exact IDs or normalized names; probabilistic scoring, such as the classic Fellegi–Sunter framework — a statistical model that weighs how much each matching or mismatching field shifts the odds that two records are the same entity, so a shared tax ID counts far more than a shared first name; fuzzy string similarity; or ML classifiers. Increasingly, embeddings and LLMs are used for semantic matching of tricky cases. - Clustering and merging groups matched records into a single canonical entity with a golden record and links to sources, where survivorship rules decide which value wins. - Identity management maintains stable canonical IDs and mappings so downstream joins are consistent.

The main options — tools. There are Master Data Management suites such as Informatica, Reltio, Tamr, and Quantexa; open frameworks such as Zingg — including on Databricks and Spark — plus Dedupe and Splink; cloud options such as AWS Entity Resolution; graph-native entity resolution; and LLM and transformer-based entity resolution for hard or edge cases. That last option is now maturing — benchmarked and appearing in platforms — but it still needs guardrails against wrong merges.

When to use it (and when not to). Invest heavily in entity resolution when data comes from many sources, such as multiple CRMs after an acquisition, or CRM plus ERP plus support; when duplicates cause real harm, such as double-counting revenue or split customer views; or when the knowledge graph's value depends on correct linkage. Match strictness to the cost of error. False merges — where two real customers

get merged — are usually worse than false splits, where one customer is left as two. So tune accordingly, and keep humans in the loop for uncertain merges.

Where it goes wrong. - Over-merging collapses two distinct entities — two people both named "John Smith," or a parent and a subsidiary. This corrupts data and can leak entitlements, letting one customer see another's data. It's dangerous. - Under-merging produces a fragmented view and double counting. - No canonical ID strategy yields resolution results that can't be reused. - Ignoring provenance means you can't audit or unwind a bad merge. - Static rules on drifting data fail; entity resolution needs ongoing maintenance.

A concrete example. After acquiring a competitor, a company has ACME in three CRMs with different spellings and IDs. Entity resolution produces one canonical ACME node linked to all three source records. Now "total ACME pipeline across both businesses" is answerable — and crucially, entitlement rules apply to the unified customer correctly, without accidentally merging ACME with a similarly-named unrelated firm.

4.5 Graph databases and query languages

Why it matters. A graph database stores and queries nodes and edges natively, making multi-hop traversals fast and expressive — it's the operational home of a knowledge graph. Relational databases can model relationships via joins, but deep or variable-length traversals — such as "all components transitively required by this product" — become painful, join-heavy queries. Graph databases make them natural.

How it works — the landscape.

For Labeled Property Graphs: - Neo4j is the most widely-used enterprise graph database. It uses the Cypher query language and has mature tooling. It now ships a native VECTOR data type with filterable vector indexes, in-Cypher GraphRAG and AI procedures, and an official neo4j-graphrag Python library — these are generally available product features, not experiments. - Others include Amazon Neptune, which supports both Labeled Property Graph and RDF; Neptune Analytics, which is in-memory with native vector search in openCypher and powers Amazon Bedrock Knowledge Bases GraphRAG, generally available in 2025; TigerGraph; Memgraph; ArangoDB, which is multi-model; and JanusGraph. The space is consolidating. For example, the embedded graph database Kùzu was acquired by Apple and its open-source repo archived in October 2025, with maintenance passing to community forks — so weigh vendor and project durability.

For RDF and triple stores, options include GraphDB, Stardog, Blazegraph, and Neptune, using SPARQL queries and OWL reasoning.

On query languages: - Cypher uses a pattern-matching syntax that effectively draws the graph in ASCII. For instance, a Cypher query can match accounts that have an opportunity whose stage is "Renewal," and return both the account and the opportunity. It's intuitive, and it's now standardized as GQL — that's ISO/IEC 39075, published in April 2024 — the first new ISO database query language since SQL in 1987. Cypher conforms to most mandatory GQL features. - SPARQL uses triple-pattern queries over RDF. It's powerful but more verbose. - Gremlin is imperative traversal, part of Apache TinkerPop, and works across vendors.

When to use it (and when not to). - The Labeled Property Graph approach, with Neo4j and Cypher, is best for most enterprise application graphs: developer-friendly, great tooling, and good LLM support for text-to-Cypher. Choose it when relationships and properties dominate and you want productivity. - Choose RDF and SPARQL for standards-based interoperability, formal ontologies and reasoning, and linked-data or regulatory contexts. - Choose multi-model stores, such as Neptune or ArangoDB, when you want graph alongside document or relational data in one store. - Also weigh scale, managed versus self-hosted, vector-index support for graph-plus-vector hybrids, and your existing skills.

Where it goes wrong. - Choosing a graph database when relational plus a few joins would do adds ops complexity for no benefit. - Unbounded traversals are risky: a match with no depth limit can explode, so bound your path lengths. - Text-to-Cypher or text-to-SPARQL errors happen when LLMs generate invalid or unsafe queries; validate, sandbox, and constrain them to read-only with row limits. - Sync drift occurs when the graph mirrors source-of-record data that changed; it needs a refresh pipeline.

A concrete example. A support-ops team runs Neo4j linking incidents to affected products to customers to contracts. Cypher answers "which enterprise-contract customers are affected by the current P1 defect?" in one traversal — feeding a proactive-outreach agent.

4.6 The semantic layer over enterprise data

This is the third strand of the system-of-record theme. It's the concept that makes structured enterprise data usable by an agent, and it recurs later in the knowledge base.

Why it matters. Enterprise data is messy: cryptic table and column names such as "acct_x2" or "opp_amt_c," duplicated across systems, encoded in ways only tribal knowledge decodes, and spread over CRM, ERP, and warehouse. A semantic layer is a business-meaning abstraction on top of raw schemas. It maps the mess to clean, governed concepts — "Customer," "Annual Recurring Revenue," "Open Pipeline" — with defined relationships, metrics, and access rules. It's the translation layer between how data is stored and how the business, and an agent, thinks.

Here's an analogy. The raw database is a warehouse of unlabeled boxes with part-number codes. The semantic layer is the catalog that says "box 4471-B is an enterprise firewall, ships with these licenses,

owned by the networking business unit." An agent reasons over the catalog, not the box codes.

How it works. - Concept and entity definitions are canonical business objects mapped to underlying tables and fields — often the knowledge graph or ontology itself. - Metric definitions are governed, single-definition calculations, such as "ARR" or "win rate," so everyone, including the agent, computes them the same way. - Relationships and joins are pre-defined so consumers don't re-derive them. - Access and governance: entitlements are defined at the semantic layer and inherited by consumers. - Two flavors are converging. First, BI and metrics semantic layers — such as the dbt Semantic Layer, Cube, LookML, and AtScale — provide governed metrics for analytics. Second, knowledge-graph and ontology semantic layers provide entities and relationships for reasoning. For agents, the semantic layer is what turns fragile "text-to-SQL against a cryptic schema" into robust "query well-defined business concepts." This shift is now benchmark-backed: 2026 dbt benchmarks show semantic-layer querying materially beating raw text-to-SQL on modeled projects, because the LLM picks a governed metric by name and the engine generates the query deterministically. Treat exact percentages as directional. - On standardization, there's a new development: the Open Semantic Interchange, or OSI, a vendor-neutral open spec backed by Snowflake, Salesforce, dbt Labs, Cube, RelationalAI, ThoughtSpot, and Atlan, announced in September 2025. It aims to make semantic definitions portable across BI tools and agents, directly targeting the "semantic layer for AI and agents" need. dbt's MetricFlow is now Apache-2.0 licensed and aligned to it.

The main options — tools. There's the dbt Semantic Layer, Cube, AtScale, and LookML in Looker. There's Palantir Foundry and AIP, whose Ontology has become a leading real-world instance of the ontology-as-agent-reasoning-backbone pattern, with strong commercial traction. There are graph and ontology platforms. And there are platform-native semantic layers — notably Salesforce Data Cloud, also called Data 360, which unifies and maps customer data into a governed model, serving as a data-unification backbone on which you may still layer governed metrics; and modeling within ServiceNow's platform, for example the CMDB, or Common Service Data Model, as its canonical model.

When to use it (and when not to). Build or adopt a semantic layer when multiple consumers — BI, apps, agents — need consistent definitions, when schemas are too messy for direct LLM querying, or when governance and entitlements must be centralized. It's often the highest-leverage investment for making agents reliable over enterprise data — and it's reusable beyond AI.

Where it goes wrong. Pitfalls include letting an agent do raw text-to-SQL over a cryptic schema, which is brittle and carries wrong-metric risk and security holes, instead of querying a governed layer; a semantic layer that drifts from the sources; defining metrics inconsistently across BI and AI, so you end up with two "revenue" numbers; and having no ownership.

A concrete example. Without a semantic layer, an agent asked for "ARR at risk this quarter" writes SQL guessing at "opp_amt_c" and renewal flags — sometimes wrong. With a semantic layer, it queries the governed ARR metric filtered by the defined "AtRisk" and "CurrentQuarter" concepts — consistent with

the CFO's dashboard, and access-controlled. This is the backbone of trustworthy source-of-record-grounded agents.

4.7 How structured knowledge integrates with LLMs

Why it matters. The payoff is combining LLMs, which give fluent reasoning, with structured knowledge, which gives precise, connected truth. LLMs supply language understanding and reasoning; graphs and semantic layers supply exact, relational, up-to-date, explainable facts. Together they cover each other's weaknesses — this is the frontier of trustworthy enterprise AI, and the heart of GraphRAG.

How it works — the main integration patterns. 1. Text-to-query, where the LLM drives the graph or database. The LLM translates a natural-language question into Cypher, SPARQL, or SQL against the semantic layer, executes it, and explains the result. Guardrails are required: validate the generated query, run it read-only with row and timeout limits, constrain it to the known schema, and never expose raw destructive access. Provide the schema or ontology in the prompt so the model queries real fields — this is where clear naming pays off. 2. Graph-as-retriever, also known as GraphRAG. Retrieve relevant subgraphs or paths, not just text passages, as context; this enables multi-hop and global questions. It's often hybrid: vector search finds entry-point entities, graph traversal expands relationships, and both feed the prompt. 3. Knowledge-graph-augmented grounding and verification. Use the graph to fact-check or constrain LLM output — for instance, does the proposed configuration satisfy the ontology's `REQUIRES` and `INCOMPATIBLE_WITH` rules? The graph is an authoritative check on the model's fluency. 4. LLM-assisted graph construction. Use LLMs to extract entities and relations from documents to build or extend the graph. This accelerates knowledge-graph creation but needs entity resolution and human review to control noise. It's now production-common — for example, Microsoft GraphRAG 1.0 and LazyGraphRAG, which defers LLM cost to query time — no longer merely emerging. 5. Ontology as an agent reasoning scaffold. The agent uses the ontology to understand valid actions and relationships before acting, which supports planning and configure-price-quote constraints.

When to use it (and when not to). Reach for structured integration when questions are relational or multi-hop, when precision and explainability are required — a graph path is an audit trail — or when the model must respect domain rules and constraints. Combine it with vector RAG rather than replacing it — text and structure answer different question types.

When sources disagree — set an authority precedence. This is a real design question once you combine structured retrieval with vector RAG. If the graph or source-of-record and the retrieved text conflict — or you've stuffed both into one prompt — which wins? Make it explicit rather than leaving it to chance. Label source authority in the prompt and rank it: authoritative structured or source-of-record facts take precedence, unstructured retrieved text is supporting context, and the model is instructed to prefer the

source-of-record record and flag the conflict rather than silently blend them. This is the generation-time complement to anchoring the graph to the source of record.

Where it goes wrong. - Text-to-query injection and errors: untrusted input coaxes a harmful query, or the LLM emits invalid or expensive queries. Sandbox, validate, apply least-privilege, run read-only, and set limits. - Schema hallucination: the model queries fields that don't exist; supply and constrain to the real schema or semantic layer. - Noisy auto-built graphs: extraction plus weak entity resolution produces wrong "facts" that now look authoritative. Review and track provenance. - Complexity and cost: graph infrastructure plus maintenance is real; don't adopt it without a relational or multi-hop need. - Two sources of truth: the graph and the source of record disagree; anchor the graph to the source of record and keep it synced.

A concrete example. A deal-desk agent is asked "can we bundle FW-9000 with the basic support tier for ACME?" First, it queries the semantic layer for ACME's entitlements and current products, using text-to-query, entitlement-filtered. Second, it checks the product ontology's REQUIRES rule and finds FW-9000 requires premium support. Third, it tells the rep the bundle is invalid and proposes the compliant alternative, citing the rule. Fluent reasoning plus structured truth gives a correct, explainable, governed answer — exactly the pattern that later modules operationalize for configure-price-quote.

Module 4 takeaways

- Knowledge graphs encode entities and relationships explicitly, enabling precise, multi-hop, explainable answers that vector RAG can't give.
- A taxonomy is a hierarchy; an ontology is a model of concepts, relationships, and rules the agent can reason over. Start minimal.
- Schema design is query-driven; model relationships as edges; name things clearly, because LLMs read your schema.
- Entity resolution — deduplication and linkage — is make-or-break and dangerous when wrong; false merges can leak entitlements.
- Graph databases — Neo4j and Cypher, RDF and SPARQL, with GQL emerging — are the operational home; guard LLM-generated queries.
- The semantic layer turns messy schemas into governed business concepts an agent can reason over — often the highest-leverage investment for reliable source-of-record-grounded AI. This is the third strand of the system-of-record theme.
- LLM plus structured knowledge gives fluent reasoning plus precise, connected, explainable truth: text-to-query, GraphRAG, verification and constraint-checking, and LLM-assisted construction — combined with, not instead of, vector RAG.